

# From Brute Force to Surgical Precision

## Mastering Search Algorithms: A deep dive into Linear vs. Binary Search.

Coding logic isn't just about making it work; it's about making it scale. This deck explores the journey from intuitive  $O(N)$  scanning to the industry-standard  $O(\log N)$  "divide and conquer" approach.

Inspired by iCodeGuru Advanced DSA Bootcamp



-1

0

3

5

9

12

# The Challenge: Find the Target in $O(\log N)$ Time

## Problem Card

### Problem Statement

Given an array of integers sorted in ascending order, write a function to search for a 'target'. If it exists, return its index. Otherwise, return -1.

### Input Data

|    |   |   |   |   |    |        |   |
|----|---|---|---|---|----|--------|---|
| -1 | 0 | 3 | 5 | 9 | 12 | target | 9 |
| 0  | 1 | 2 | 3 | 4 | 5  |        |   |

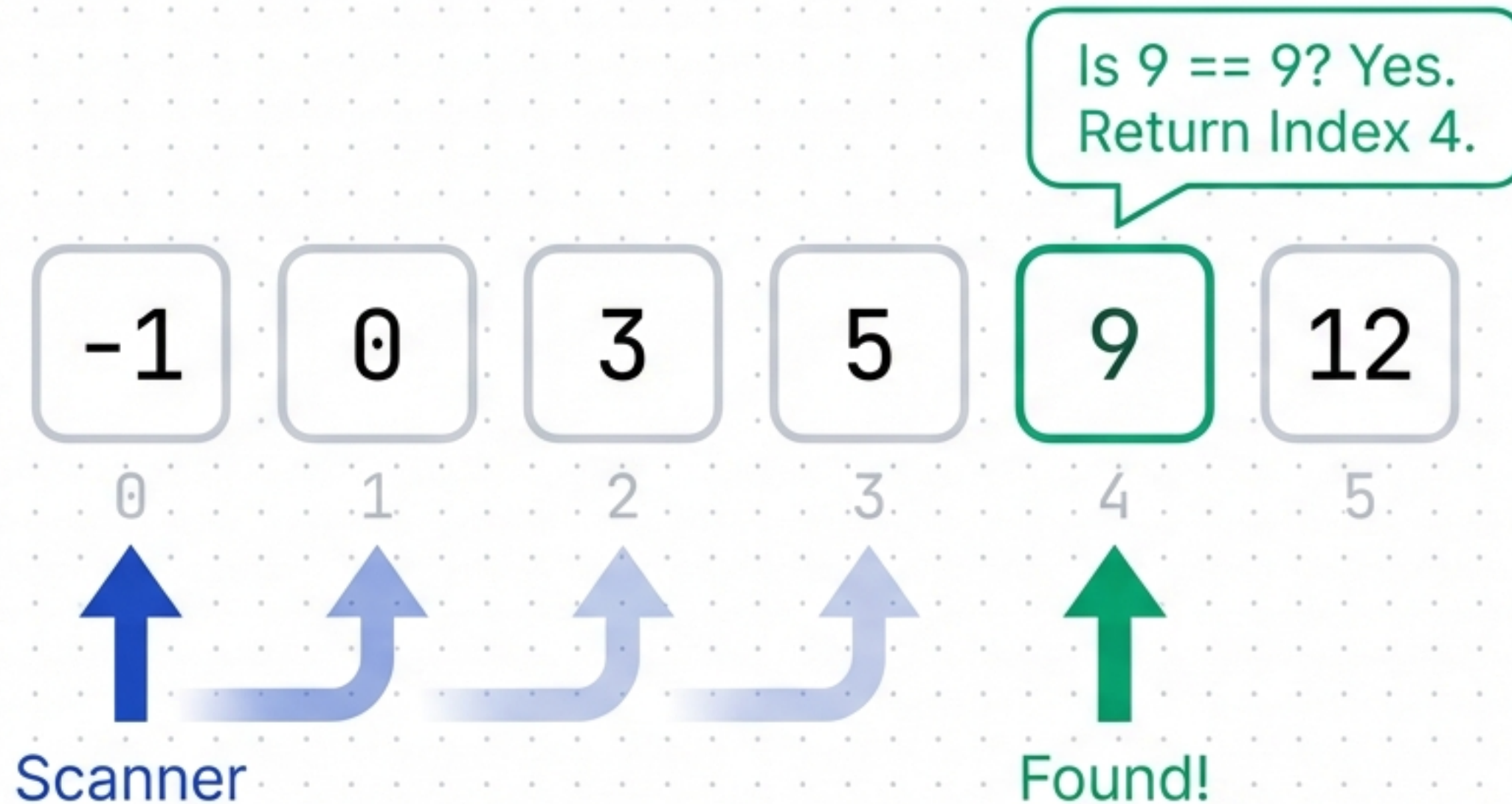


**CONSTRAINT:** You must write an algorithm with  $O(\log N)$  runtime complexity.

Finding the number is easy. finding it fast is the engineering challenge.

# The Intuitive Approach: Linear Search

Also known as “Brute Force”. We check every element one by one.



Step 1: Check Index 0  
(-1 vs 9). No.

Step 2: Check Index 1  
(0 vs 9). No.

...

Step 5: Check Index 4  
(9 vs 9). Yes.

“If the logic is clear, the code will follow. This method works, but checks every item in the worst case.”

# Coding the Linear Solution

```
def search(nums, target):  
    # Iterate through every item  
    for i, value in enumerate(nums):  
        if value == target:  
            return i  
    return -1
```

O(N) Complexity - Runs 'N' times in worst case.

## Mechanism

The code utilizes 'enumerate' to traverse the list index by index.

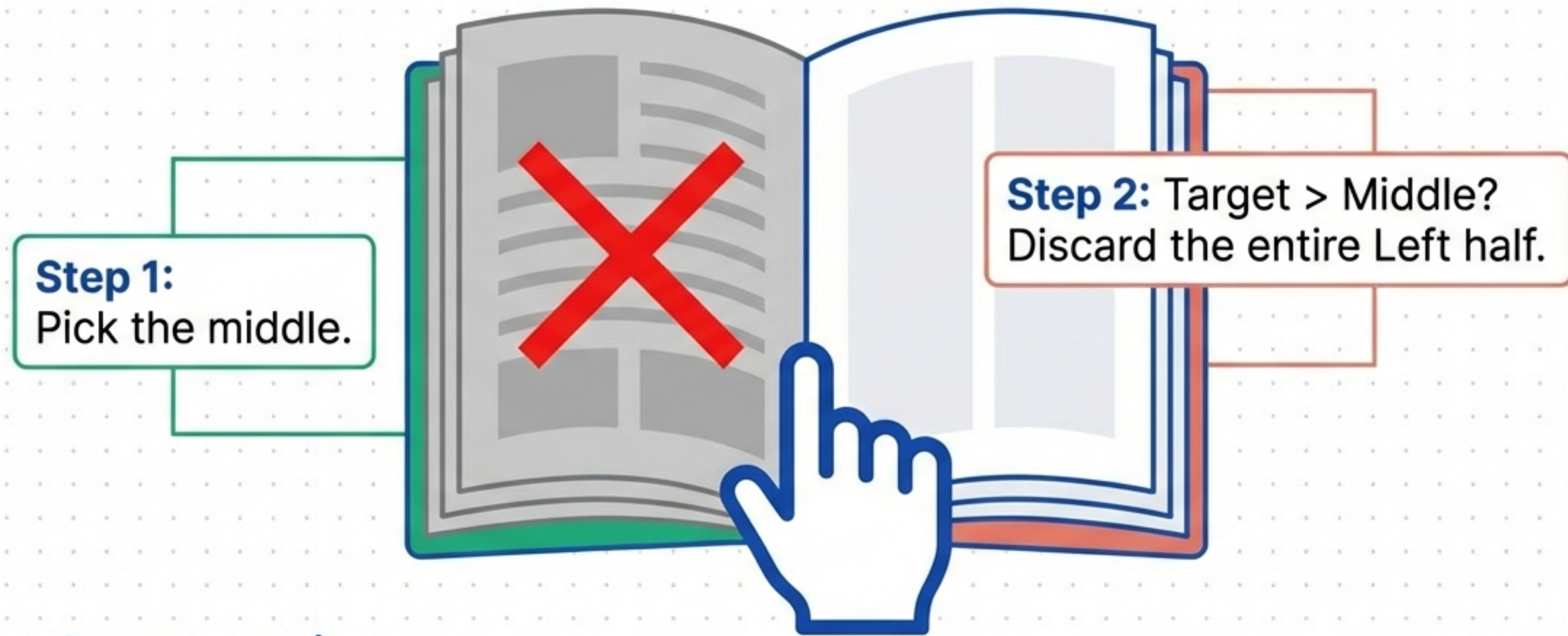
## The Flaw

If the array has 1 million items and the target is at the end (or missing), this loop runs 1 million times.

## Verdict

Valid logic, but fails the strict efficiency requirement of the problem.

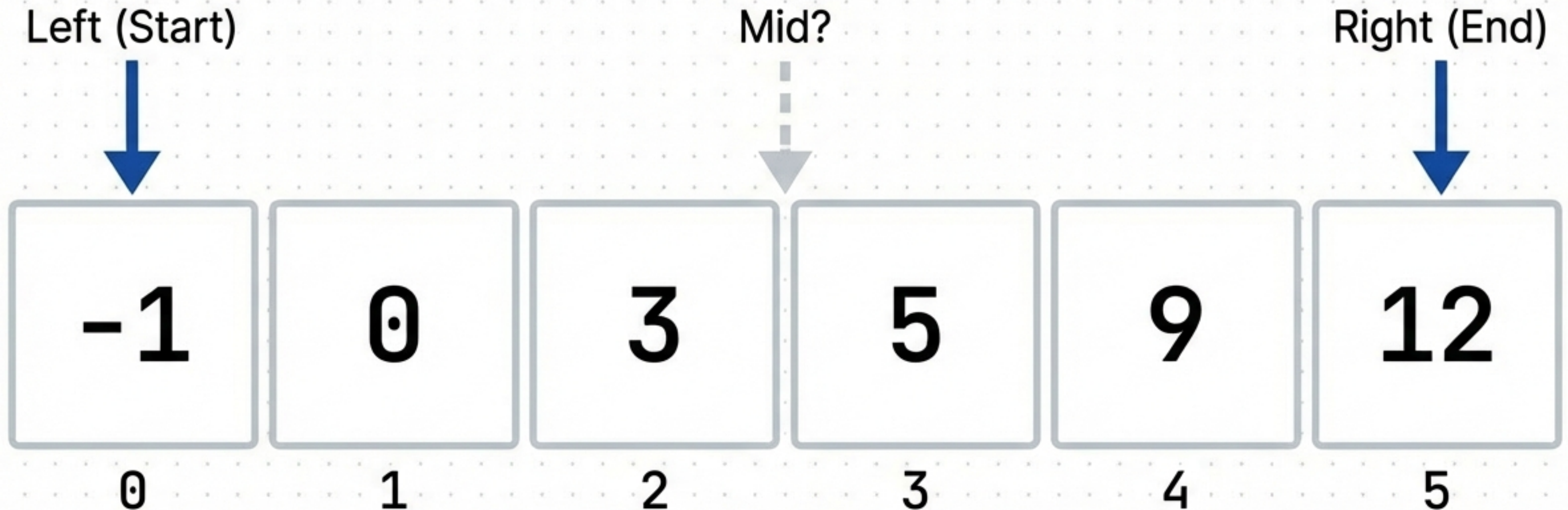
# The Paradigm Shift: Divide and Conquer



## Binary Search Logic

This approach only works on SORTED arrays. Instead of checking elements, we use the sort order to eliminate half the possibilities instantly. It's like opening a phonebook to the middle—if you need 'T' and you land on 'M', you throw away 'A–M' immediately.

# The Setup: Two Pointers and a Middle



**Left Pointer:** Starts at index 0.

**Right Pointer:** Starts at index `len(nums) - 1` (Index 5).

**Mid Pointer:** Calculated dynamically based on Left and Right.

# Calculating the Midpoint

## Avoiding Floats with Integer Division

$$\text{Left (0)} + \text{Right (5)} = 5$$

~~$$5 / 2 = 2.5$$~~

**Float Index (Invalid)**

**Python Syntax:**

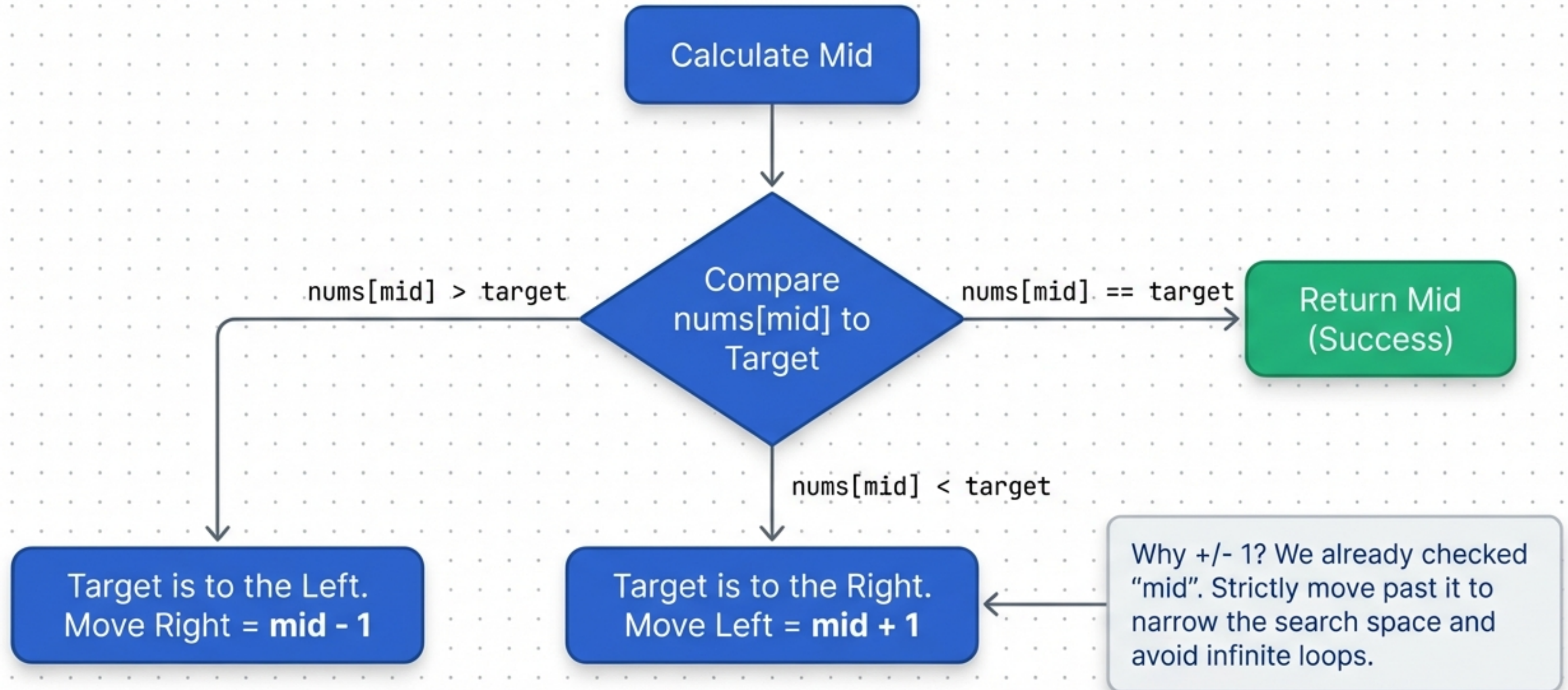
```
mid = (left + right) // 2
```

$$5 // 2 = 2 \checkmark$$

**Floor Division (Valid)**

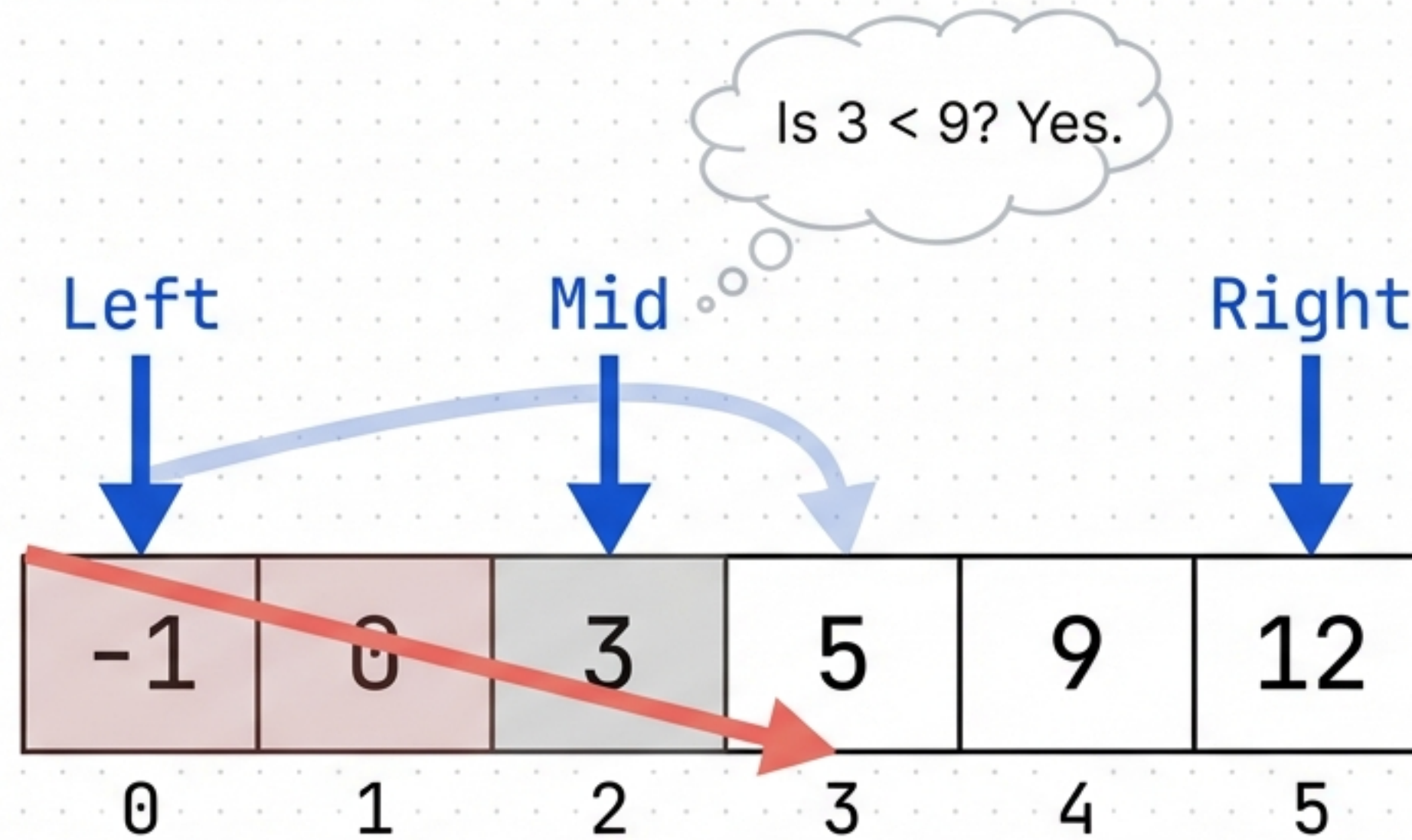
Indices cannot be decimals. We use the floor operator `//` to round down to the nearest integer, consistent with integer division in languages like Java and Python.

# The Execution Logic



# Trace: Iteration 1

Target = 9



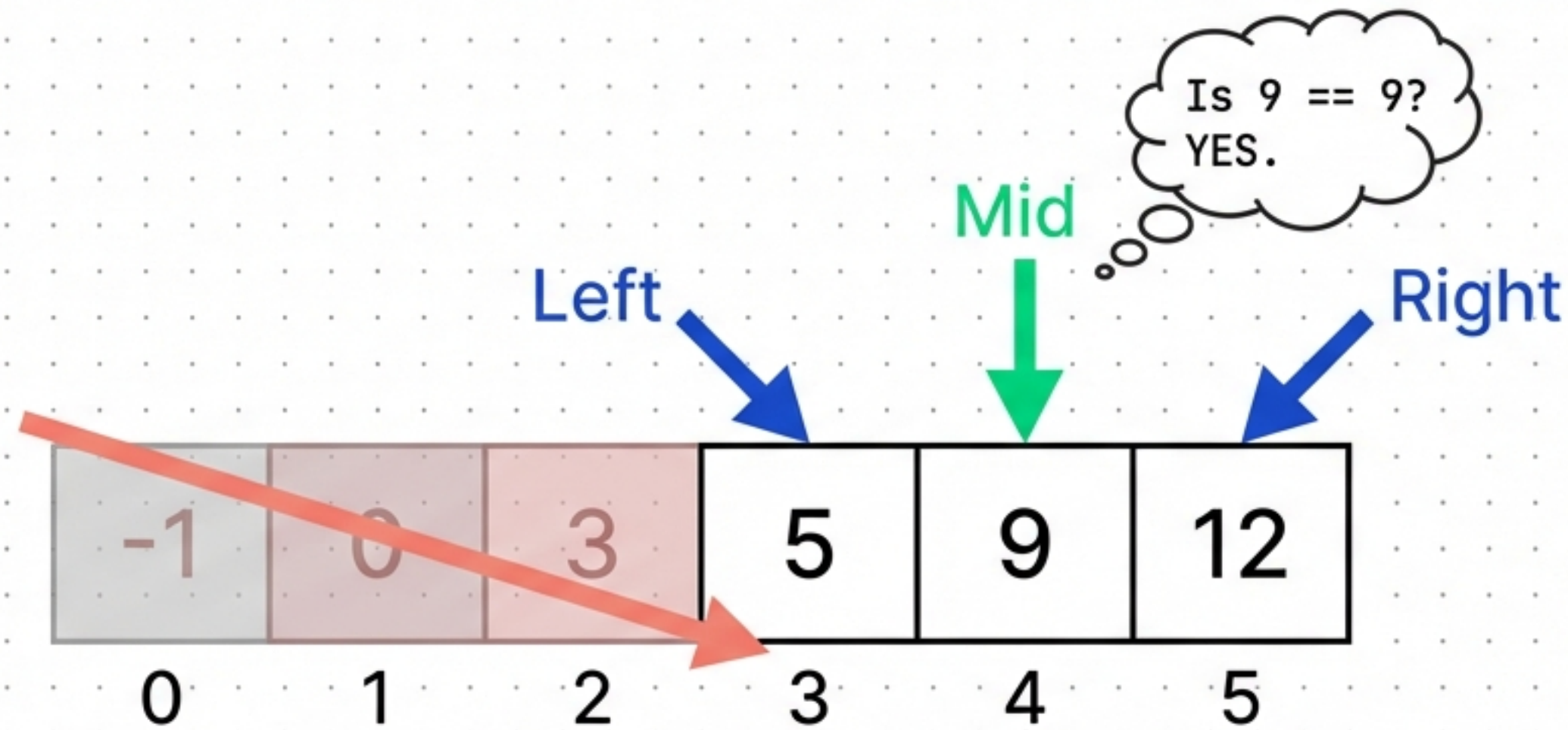
State: Left=0, Right=5, Mid=2. Value=3.

Logic:  $3 < 9$ . Target is in the right half.

Action: Left moves to  $\text{mid} + 1$  (Index 3).

# Trace: Iteration 2

Target = 9



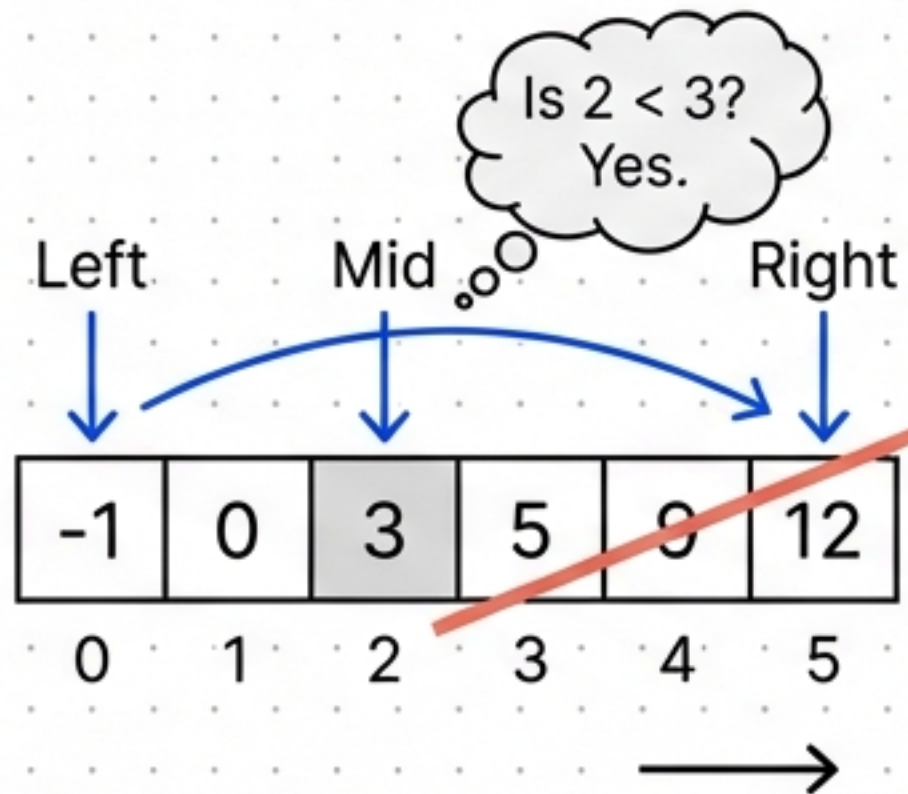
State: Left=3, Right=5.  $\text{Mid} = (3+5)//2 = 4$ . Value=9.  
Logic: Match Found.  
Result: Return 4.

Comparison Note: Linear Search steps: 5.  
Binary Search steps: 2.

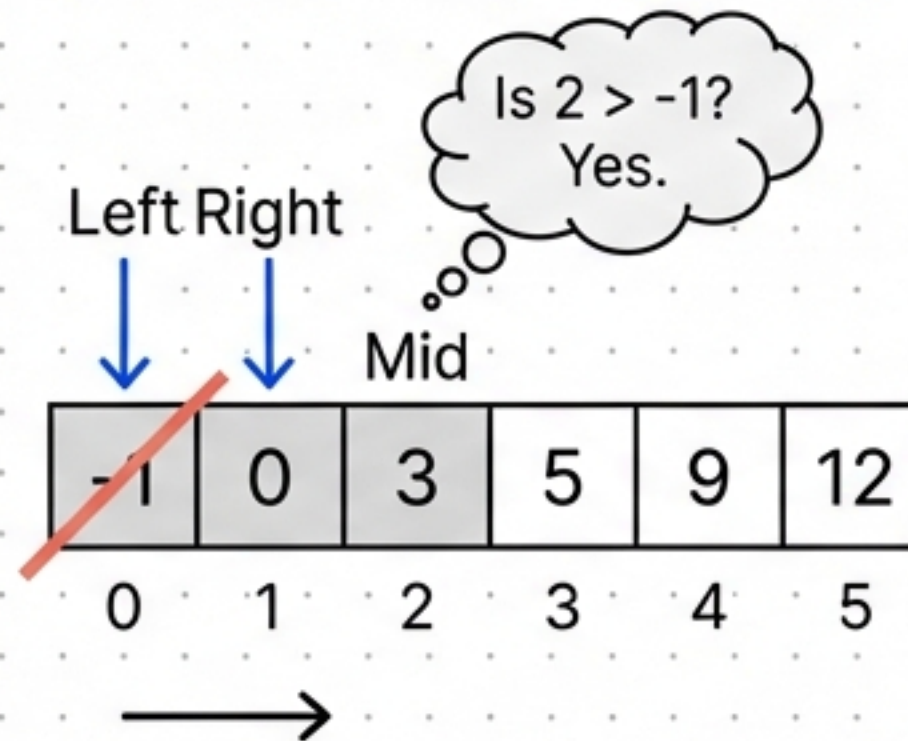
# Edge Case: Target Not Found

Target = 2

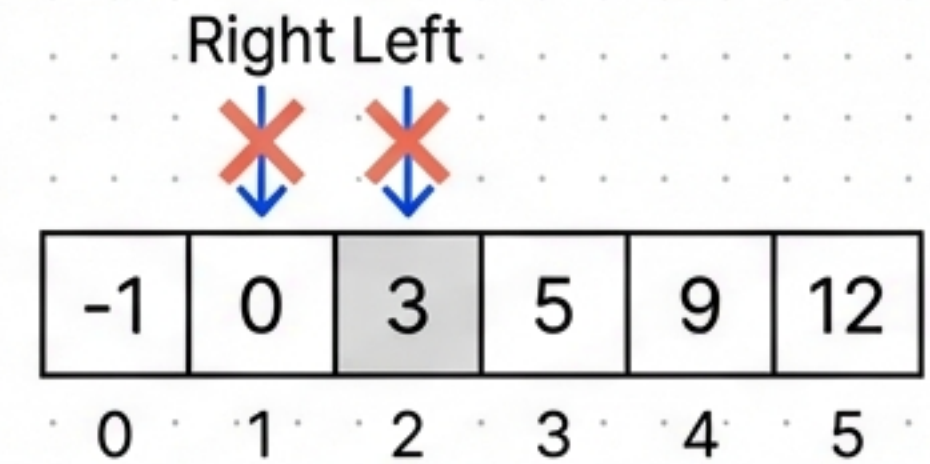
Array = `[-1, 0, 3, 5, 9, 12]`



Right moves to  
Mid - 1 (Index 1).



Left moves to  
Mid + 1 (Index 1).



**Left (2) > Right (1).  
Loop Terminates.**

**Return -1**

# The Optimized Implementation

```
def binary_search(nums, target):  
    left, right = 0, len(nums) - 1
```

**Loop Condition:** Runs until pointers cross.

```
    while left <= right:  
        mid = (left + right) // 2
```

**Integer Division:**  
Flooring the index.

```
        if nums[mid] == target:  
            return mid
```

```
        elif nums[mid] < target:  
            left = mid + 1
```

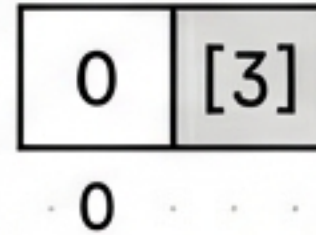
```
        else:  
            right = mid - 1
```

**Narrowing:** Excluding the checked mid.

```
    return -1
```

# Critical Detail: Why ' $\leq$ ' and not ' $<$ '?

## The Single Element Trap



Target = 3

## Incorrect Logic

```
while left < right
    mid = (left + right) // 2
    Left=0 ↘ ↙ Right=0
           0
```

**Logic:** ' $0 < 0$ ' is FALSE. Loop never starts.

**Result:** Returns -1 (Wrong).

## Correct Logic

```
while left <= right
    mid = (left + right) // 2
    Left=0 ↘ ↙ Right=0
           0
```

**Logic:** ' $0 \leq 0$ ' is TRUE. Loop runs.

**Check:** ' $nums[0] == 3$ '. Found!

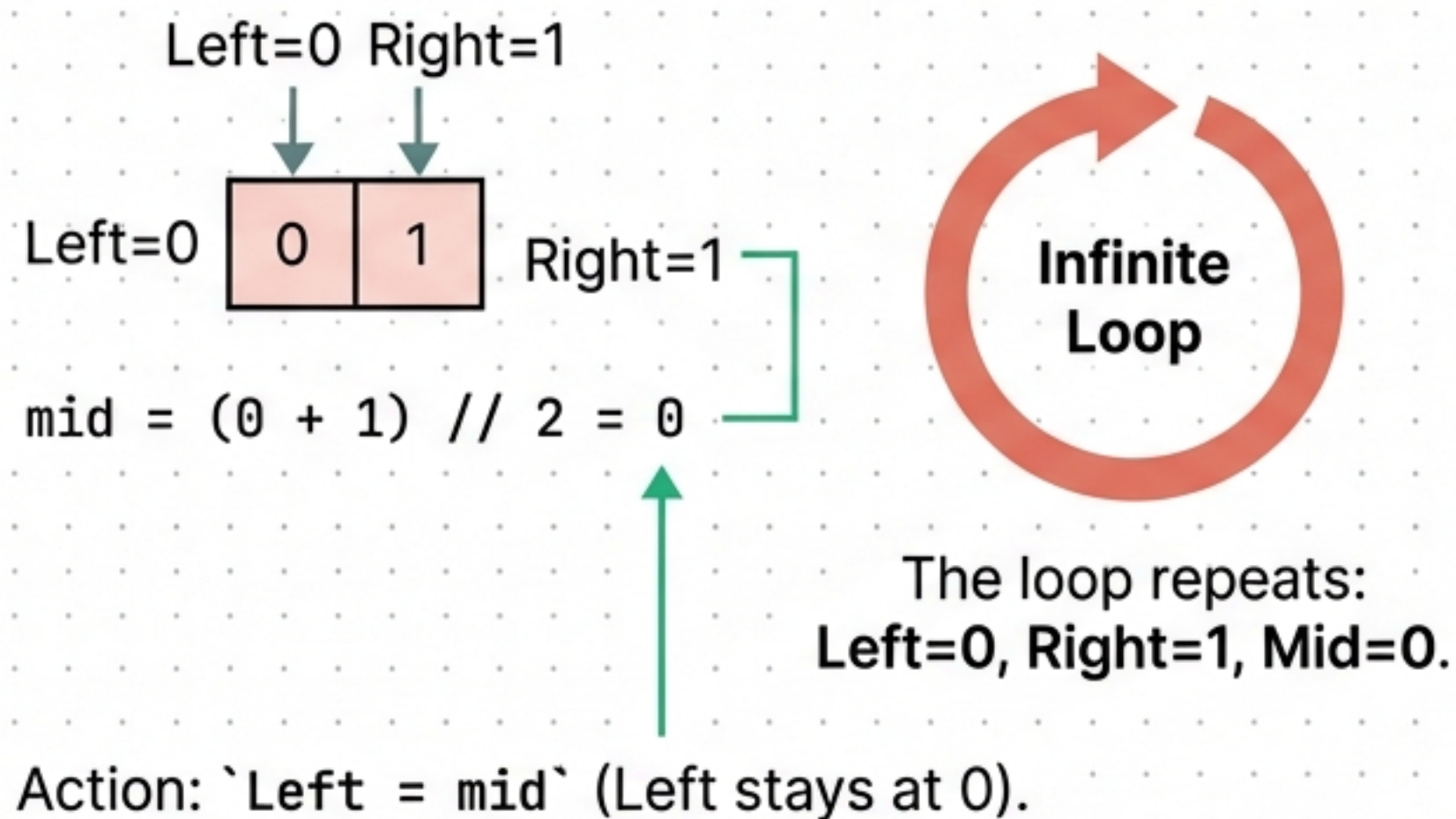
**Result:** Returns 0 (Correct).

Always handle the case where Left and Right converge on the same element.

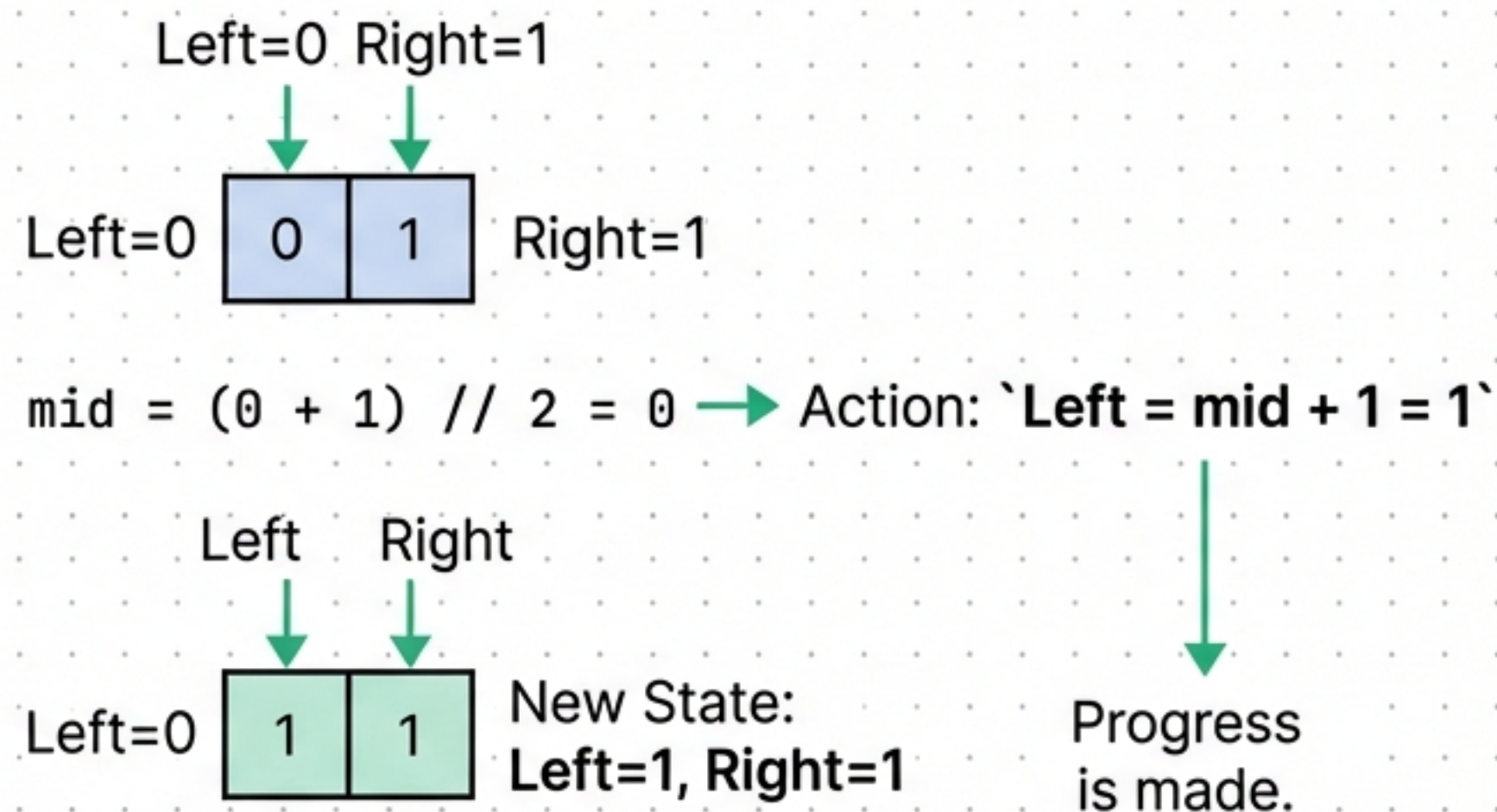
# Critical Detail: Avoiding Infinite Loops

Why do we move pointers to `mid + 1` or `mid - 1` instead of just `mid`?

## The Stall: Incorrect Logic



## The Correct Path: Making Progress



Source: JetBrains: "We discard the previous values... move the Left pointer to `Mid + 1`."

# Summary: The Power of Logarithmic Time

| Feature         | Linear Search              | Binary Search           |
|-----------------|----------------------------|-------------------------|
| Time Complexity | $O(N)$                     | $O(\log N)$             |
| Method          | Sequential Scanning        | Divide and Conquer      |
| Prerequisite    | None (works on any list)   | <b>Sorted Data Only</b> |
| Best For        | Unsorted or small datasets | Large, sorted datasets  |

**When working with massive datasets, we don't just need to find the answer; we need to find it before the user blinks. Binary search turns a mountain of data into a manageable handful of steps.**